

Chapter 3 Notes: Classes and Objects

Object-Oriented Analysis and Design Study Notes

Contents

1	Chapter Purpose	2
2	The Nature of an Object	2
2.1	What Is an Object?	2
2.2	What Is Not an Object?	2
2.3	Objects and Boundaries	3
2.4	State	3
2.4.1	Static Properties Versus Dynamic Values	3
2.4.2	Objects Versus Simple Values	4
2.4.3	Encapsulating State	4
2.5	Behavior	5
2.5.1	Operation, Method, Member Function, and Message	5
2.5.2	Behavior Depends on State	5
2.6	Kinds of Operations	5
2.7	Protocol, Roles, and Responsibilities	6
2.8	Objects as Machines	6
2.8.1	Finite State Machine View	6
2.8.2	Active and Passive Objects	7
2.9	Identity	7
2.9.1	Identity Is Not the Same as Name	7
2.9.2	Structural Sharing and Aliasing	8
2.9.3	Engineering Rule for Identity	8
3	Relationships Among Objects	8
3.1	Why Object Relationships Matter	8
3.2	Links	9
3.2.1	Client and Supplier	9
3.2.2	Roles in a Link	9
3.3	Visibility Across Links	9
3.4	Synchronization	10
3.5	Aggregation Among Objects	10
3.6	Links Versus Aggregation	11
4	The Nature of a Class	11
4.1	What Is a Class?	11
4.2	Class Versus Object	11
4.3	Classes Are Necessary but Not Always Sufficient	12
4.4	Interface and Implementation	12

4.5	Visibility Levels	12
4.5.1	Friendship and Package Access	12
4.6	Class Lifecycle Semantics	13
5	Relationships Among Classes	13
5.1	Overview	13
5.2	Association	13
5.2.1	When to Use Association	13
5.2.2	Multiplicity	14
5.2.3	Association Design Mistakes	14
5.3	Inheritance	14
5.3.1	The Is-A Litmus Test	14
5.3.2	Inheritance for Extension and Restriction	15
5.4	Abstract and Concrete Classes	15
5.5	Polymorphism	16
5.5.1	Why Polymorphism Matters	16
5.6	Overloading Versus Polymorphism	16
5.7	Dynamic Dispatch	17
5.8	Single Inheritance	17
5.9	Multiple Inheritance	18
5.9.1	Problems in Multiple Inheritance	18
5.9.2	Mixins	18
5.10	Inheritance Versus Encapsulation	18
5.11	Aggregation Among Classes	19
5.11.1	Containment by Value and by Reference	19
5.12	Dependencies	19
6	The Interplay of Classes and Objects	20
6.1	Static Classes, Dynamic Objects	20
6.2	Key Abstractions and Mechanisms	20
6.3	Outside View Versus Inside View	21
7	Building Quality Classes and Objects	21
7.1	Iterative Nature of OO Design	21
7.2	Quality Criteria for Abstractions	21
7.3	Coupling	22
7.3.1	Coupling and Inheritance	22
7.4	Cohesion	22
7.5	Sufficiency	22
7.6	Completeness	22
7.7	Primitiveness	23
7.8	Choosing Operations	23
7.8.1	Fine-Grained Versus Coarse-Grained Operations	23
7.8.2	Where Should a Behavior Live?	23
7.9	Time and Space Semantics	24
7.10	Choosing Relationships	24
7.10.1	Law of Demeter	24
7.10.2	Wide-Shallow Versus Narrow-Deep Inheritance	25

7.11	Inheritance, Aggregation, and Dependency Decision Guide	25
7.12	Mechanisms and Visibility	25
7.13	Choosing Implementations	26
7.13.1	Representation as a Secret	26
7.14	Packaging	26
8	UML and Diagram Interpretation	27
8.1	Class Diagrams	27
8.2	Object Diagrams	27
8.3	State Machine Diagrams	28
8.4	Common UML Mistakes	28
9	Key Takeaways	28
10	Common Pitfalls	29
10.1	Pitfall 1: Confusing Object Identity with Equality	29
10.2	Pitfall 2: Exposing Representation	29
10.3	Pitfall 3: Using Inheritance for Code Reuse Alone	29
10.4	Pitfall 4: Creating Fat Interfaces	29
10.5	Pitfall 5: Ignoring Lifecycle Semantics	29
10.6	Pitfall 6: Overusing Bidirectional Links	29
10.7	Pitfall 7: Forgetting Synchronization	29
10.8	Pitfall 8: Violating the Law of Demeter	29
11	Interview Questions	29
11.1	Objects and Classes	29
11.2	Behavior and Operations	30
11.3	Relationships Among Objects	30
11.4	Class Relationships	30
11.5	Quality of Abstractions	31
12	Engineering Insights	31
12.1	Objects Are Responsibility Holders, Not Data Bags	31
12.2	Polymorphism Supports Open/Closed Design	32
12.3	Encapsulation Enables Representation Change	32
12.4	Inheritance Is a Semantic Commitment	32
12.5	Object Design Is Also Concurrency Design	32
12.6	Mechanisms Are the Heart of OO Architecture	32
13	Connections to Other Topics	32
13.1	Connection to Abstraction	32
13.2	Connection to Encapsulation	32
13.3	Connection to Modularity	32
13.4	Connection to SOLID Principles	32
13.5	Connection to Design Patterns	33
13.6	Connection to System Design	33

14 Practical Applications	33
14.1 Domain Modeling	33
14.2 API Design	33
14.3 Refactoring	34
14.4 Testing	34
14.5 Architecture Documentation	34
15 Summary Cheat Sheet	34
15.1 Core Definitions	34
15.2 Relationship Decision Table	35
15.3 Design Heuristics	35
15.4 Exam Memory Hooks	35

1 Chapter Purpose

Object-oriented analysis and design uses **classes** and **objects** as the basic building blocks for modeling, decomposing, and implementing complex systems. This chapter explains what objects and classes are, how they relate, and how engineers judge whether the resulting abstractions are high quality.

Core Thesis

An object-oriented system is not merely a set of data structures plus procedures. It is a society of collaborating objects. Each object has state, behavior, and identity; each class defines the common structure, behavior, and meaning shared by similar objects.

The practical goal is to learn how to:

- distinguish objects from classes;
- identify state, behavior, and identity;
- use object and class relationships correctly;
- avoid common design errors such as exposing representation, abusing inheritance, and over-coupling objects;
- build abstractions that are cohesive, sufficient, complete, primitive, and loosely coupled.

2 The Nature of an Object

2.1 What Is an Object?

An **object** is an entity that has:

1. **state**: what the object currently knows or contains;
2. **behavior**: what the object can do or how it reacts;
3. **identity**: what makes the object distinct from every other object.

Definition

An object is an entity that has state, behavior, and identity. The structure and behavior of similar objects are defined in their common class. The terms **object** and **instance** are often used interchangeably.

Objects may represent:

- tangible things, such as machines, employees, sensors, bank accounts, and files;
- conceptual things, such as orders, reservations, transactions, workflows, and chemical processes;
- design inventions, such as controllers, adapters, proxies, services, schedulers, and repositories.

2.2 What Is Not an Object?

Not every noun should become an object. Many candidate nouns are better modeled as attributes, values, roles, or relationships.

Candidate	Usually Not an Object When	Better Modeling Choice
Color	It only describes another thing	Attribute, such as <code>color</code>
Beauty	It is a subjective quality	Attribute, metric, or domain rule
Love	It describes a relationship or emotion	Relationship, state, or event
Salary amount	It is a simple value	Value object or primitive value
“Current floor”	It is just one property of an elevator	Attribute of <code>Elevator</code>

Engineering Rule

A useful object has a meaningful boundary, a role in the problem domain, and behavior worth encapsulating. A word appearing in requirements is only a candidate abstraction, not proof that it deserves a class.

2.3 Objects and Boundaries

Objects usually have boundaries. These boundaries may be:

- **physical:** a vending machine, elevator, airplane wing, server rack;
- **conceptual:** a transaction, route, chemical process, temperature ramp;
- **fuzzy at low levels but useful at high levels:** a fog bank, crowd, river, market segment.

The boundary must be appropriate to the abstraction level. A fog bank may not be useful as a separate object to a person walking through it, but it may be useful to a weather system.

2.4 State

State is the collection of properties of an object plus the current values of those properties.

Definition

The state of an object encompasses all of the usually static properties of the object plus the current usually dynamic values of those properties.

For a vending machine, state may include:

- amount of money currently inserted;
- inventory of drinks;
- amount of change available;
- whether exact change is required;
- current operational status.

The order of operations matters because state changes over time. Inserting money and then pressing a button is not equivalent to pressing a button and then inserting money.

2.4.1 Static Properties Versus Dynamic Values

Concept	Meaning	Example
Property	Inherent characteristic of an object	A vending machine accepts money
Dynamic value	Current value of a property	The inserted amount is \$1.25
Static value	Value that normally does not change	Serial number of the machine
State transition	Change caused by an operation or event	Inventory decreases after vending

2.4.2 Objects Versus Simple Values

Objects and values are not the same.

Simple Value	Object	Design Implication
Atemporal	Exists over time	Objects have lifecycles
Usually immutable	Usually mutable	Objects may change state
No identity beyond value	Has unique identity	Two objects may have equal state but different identity
Not instantiated in the OO sense	Created and destroyed	Object lifecycle must be managed
Example: 3	Example: Elevator	Use values for quantities; objects for entities

2.4.3 Encapsulating State

Exposing state directly makes every client dependent on the internal representation of the object. Good design hides state and exposes behavior.

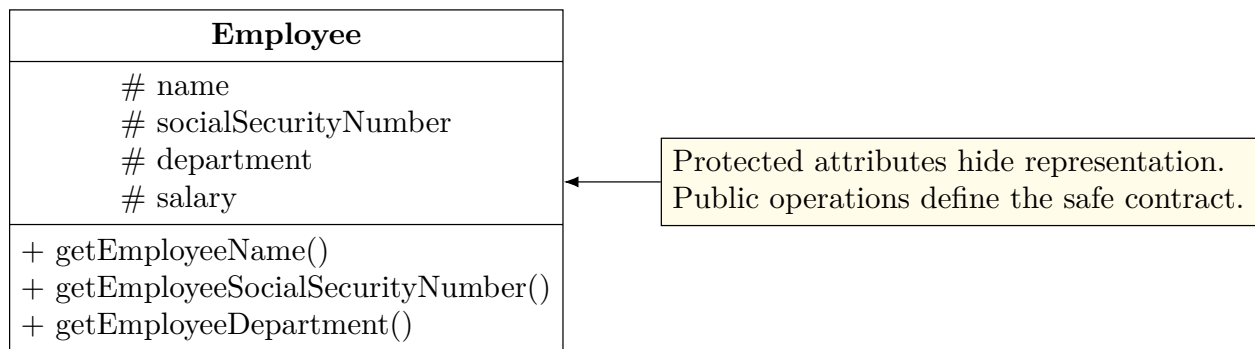


Figure 1: A class should protect representation and expose stable operations.

Why Encapsulation Matters

If the internal representation of **Employee** changes, clients that use public operations can remain semantically unaffected. If clients directly read and write fields, every representation change can break external code.

2.5 Behavior

Behavior is how an object acts and reacts in terms of state changes and message passing.

Definition

Behavior is how an object acts and reacts, in terms of its state changes and message passing.

Behavior is outwardly visible. Clients should care about what the object promises to do, not how it internally represents its data.

2.5.1 Operation, Method, Member Function, and Message

Different languages use different vocabulary:

- **operation**: an abstract service that can be requested;
- **method**: a language-level implementation of an operation, common in Java and Smalltalk terminology;
- **member function**: C++ terminology for a method;
- **message**: a request sent from one object to another.

For design purposes, these ideas are closely related: one object asks another object to perform a service.

2.5.2 Behavior Depends on State

The same operation can produce different results depending on current state. For example, `selectDrink()` on a vending machine may:

- do nothing if not enough money has been inserted;
- vend the drink if the item is available and payment is sufficient;
- return an error if the item is sold out;
- refuse the operation if the machine is out of service.

Important Refinement

The state of an object represents the cumulative results of its behavior.

2.6 Kinds of Operations

A class usually offers several categories of operations.

Operation Type	Purpose	Example
Modifier	Alters object state	<code>deposit(amount)</code> , <code>moveTo(x,y)</code>
Selector	Reads state without changing it	<code>balance()</code> , <code>location()</code>
Iterator	Accesses parts in a defined order	<code>iterator()</code> , <code>next()</code>
Constructor	Creates and initializes an object	<code>new Account(owner)</code>
Destructor	Frees resources or destroys object	C++ destructor, explicit close method

Listing 1: Example operations on a queue abstraction

```
public interface Queue<E> {
    void enqueue(E item); // modifier
    E dequeue(); // modifier and selector
    E peek(); // selector
    int size(); // selector
    Iterator<E> iterator(); // iterator
}
```

2.7 Protocol, Roles, and Responsibilities

The **protocol** of an object is the set of operations that clients may use. A protocol defines the envelope of legal behavior.

A **role** is a logical grouping of behavior that an object presents to a particular kind of client. A **responsibility** is a service or obligation the object provides as part of a role.

Concept	Meaning	Example
Protocol	All operations exposed by an object	Account.deposit, withdraw, balance
Role	A client-relative view of behavior	Account as monetary asset; account as taxable entity
Responsibility	Service the object is expected to provide	Maintain balance invariant; report interest
Contract	Promise between abstraction and client	“Withdraw either succeeds or reports insufficient funds”

Objects can play several roles during their lifetimes. A person may be a doctor, parent, gardener, and reviewer. A stock may be a tradable asset to a trader and a legal instrument to a lawyer.

Design Insight

Analyzing roles is often easier than immediately designing methods. First ask, “What responsibilities does this object have to different clients?” Then refine those responsibilities into operations.

2.8 Objects as Machines

Because state matters, the order of operations matters. Many objects can be viewed as small machines whose behavior depends on their current state and incoming events.

2.8.1 Finite State Machine View

An object may be modeled as a finite state machine when:

- it has a small number of meaningful states;
- certain operations are valid only in certain states;
- event ordering matters;
- the object must enforce lifecycle rules.

Examples include:

- **Order:** created, paid, shipped, delivered, canceled;
- **Connection:** closed, opening, open, closing, failed;
- **Elevator:** idle, moving up, moving down, door open, emergency stop.

2.8.2 Active and Passive Objects

Object Type	Meaning	Example
Active object	Has its own thread of control and may act autonomously	Event loop, scheduler, actor, server worker
Passive object	Changes only when acted upon by another object	Value holder, entity object, collection
Controller object	Initiates operations on others	FlowController , UI controller
Server object	Is operated on by others	DisplayPanel , repository
Proxy object	Represents another domain object and may both call and be called	Sensor proxy, remote service proxy

Architecture Implication

Active objects often become roots of control. In a concurrent architecture, several active objects may coordinate through messages, locks, queues, or actors. In a sequential architecture, one main active object may dispatch events to many passive objects.

2.9 Identity

Identity is the property that distinguishes one object from all other objects.

Two objects may:

- have different identities but equal state;
- have the same identity even after their state changes;
- be referenced by multiple names or aliases;
- lose all references and become unreachable.

2.9.1 Identity Is Not the Same as Name

A variable name is not the object. A variable may hold a reference to an object; multiple variables may refer to the same object.

Listing 2: Object identity and aliasing

```
DisplayItem item1 = new DisplayItem(0, 0);
DisplayItem item2 = new DisplayItem(0, 0);

System.out.println(item1 == item2); // false: different identity
System.out.println(item1.equals(item2)); // may be true if equality compares state

DisplayItem item3 = item2; // alias: same object as item2
item3.moveTo(75, 75);
```

```
System.out.println(item2.location()); // also (75, 75)
```

2.9.2 Structural Sharing and Aliasing

Structural sharing occurs when more than one reference names the same object. This is useful but dangerous.

Common Bugs Caused by Aliasing

- Unexpected state change: modifying through one reference changes what another reference sees.
- Dangling reference: a reference points to an object that has been destroyed or invalidated.
- Memory leak: an object remains reachable longer than intended.
- Representation leak: clients gain access to mutable internal objects.

2.9.3 Engineering Rule for Identity

Use value equality when comparing values; use identity equality when asking whether two references denote the same object.

Question	Concept	Example
Are these the same object?	Identity equality	<code>a == b</code> in Java for references
Do these objects represent the same value?	Logical equality	<code>a.equals(b)</code>
Can this object change?	Mutability	<code>ArrayList</code> is mutable
Can many clients safely share it?	Aliasing risk	Immutable objects are safer to share

3 Relationships Among Objects

3.1 Why Object Relationships Matter

An object in isolation is usually uninteresting. System behavior emerges from collaborations among objects.

Core Idea

Object-oriented design replaces a single centralized procedure manipulating passive data with a network of objects that request services from one another.

The relationship between two objects includes the assumptions each makes about:

- which operations may be invoked;
- what behavior will result;
- what state may be observed or changed;
- what visibility or reference is required;

- what synchronization is needed.

The two major object relationships are:

1. **links**;
2. **aggregation**.

3.2 Links

A **link** is a physical or conceptual connection between objects. A link allows one object to navigate to another or request services from another.

Definition

A link denotes the specific association through which one object, the client, applies the services of another object, the supplier.

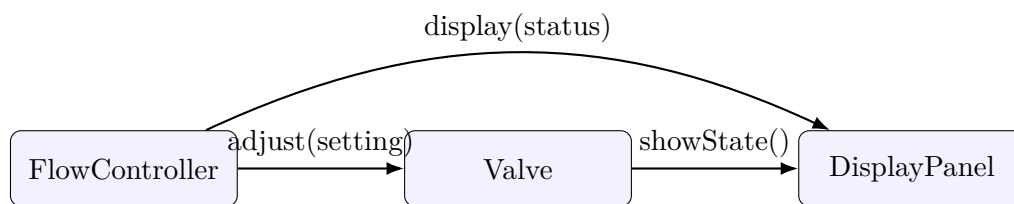


Figure 2: Links support message passing among collaborating objects.

3.2.1 Client and Supplier

- The **client** sends a message.
- The **supplier** receives the message and provides the requested behavior.
- Message direction and data flow are not identical: a client may send arguments to a supplier and receive a result back.

3.2.2 Roles in a Link

Role	Meaning	Example
Controller	Operates on other objects but is not operated on by them in that collaboration	FlowController
Server	Is operated on but does not operate on others	DisplayPanel
Proxy	Both operates on others and is operated on; often represents a real-world object	Valve proxying a physical valve

3.3 Visibility Across Links

Object A can send a message to object B only if B is visible to A. Visibility is often ignored during early analysis but becomes crucial during implementation.

Typical ways an object becomes visible:

- **field visibility:** A stores a reference to B;
- **parameter visibility:** B is passed into one of A's methods;
- **local visibility:** A creates or retrieves B inside a method;
- **global visibility:** B is globally accessible, which is usually risky.

Engineering Tradeoff

More visibility makes collaboration easier but increases coupling. Less visibility improves information hiding but may require explicit coordination mechanisms such as factories, dependency injection, event buses, or service registries.

3.4 Synchronization

When one object sends a message to another, the two objects are synchronized in some way.

Synchronization	Meaning	Typical Use
Sequential	Supplier semantics are safe only with one active client at a time	Single-threaded systems
Guarded	Supplier can be used by multiple threads only if clients cooperate to ensure mutual exclusion	Client-managed locks
Concurrent	Supplier guarantees its own thread safety	Thread-safe queue, actor mailbox

Concurrency Pitfall

A passive object that is safe in a sequential program may be unsafe when several active objects access it concurrently. Thread safety is a design property, not something automatically provided by object orientation.

3.5 Aggregation Among Objects

Aggregation is a whole/part relationship among objects. Unlike a simple link, aggregation implies that one object is an aggregate and another is one of its parts.

Examples:

- airplane has wings, engines, and landing gear;
- temperature controller has a heater and a temperature ramp;
- shareholder has shares as part of the shareholder's assets;
- document has paragraphs and figures.

Definition

Aggregation denotes a whole/part hierarchy, with the ability to navigate from the whole to its parts.

Aggregation may be physical or conceptual:

- **physical containment:** an airplane contains an engine;
- **conceptual containment:** a shareholder owns shares, but does not physically contain them.

3.6 Links Versus Aggregation

Question	Use Link When	Use Aggregation When
Relationship meaning	Objects collaborate as peers or client/supplier	One object is naturally part of another
Lifecycle	Lifetimes are independent	Whole may manage or constrain part lifetime
Coupling	Looser coupling is preferred	Encapsulating parts inside whole is preferred
Navigation	Either object may know the other as needed	Whole-to-part navigation is central
Example	Controller uses repository	Order contains order lines

Design Insight

Aggregation can hide parts as secrets of the whole, improving encapsulation. Links can preserve looser coupling. The right choice depends on whether the domain relationship is truly whole/part.

4 The Nature of a Class

4.1 What Is a Class?

A class is not a concrete object. A class is an abstraction describing the common structure, behavior, and semantics of a set of objects.

Definition

A class is a set of objects that share a common structure, common behavior, and common semantics.

For example:

- `Mammal` is a class;
- a specific dog named Luna is an object;
- `BankAccount` is a class;
- account number 12345 owned by a particular customer is an object.

4.2 Class Versus Object

Aspect	Class	Object
Nature	Abstraction/type	Concrete instance
Existence	Usually static in program structure	Created and destroyed at runtime
Represents	Common structure and behavior	Specific entity with state
Example	<code>Employee</code>	Employee object for Tom

Quantity	One class may have zero or many instances	Each object belongs to a class
UML notation	Class box with name, attributes, operations	Object box often underlined

4.3 Classes Are Necessary but Not Always Sufficient

Some abstractions are too large to be represented by a single class. A GUI framework, database, or inventory subsystem may be conceptually one thing at a high level, but it is usually implemented as a cluster of collaborating classes.

Fat Class Warning

Do not force a large subsystem into one giant class. A fat interface creates coarse reuse, high coupling, brittle clients, and excessive recompilation when small details change.

4.4 Interface and Implementation

A class has an outside view and an inside view.

- The **interface** is the outside view. It declares what clients may rely on.
- The **implementation** is the inside view. It contains representation and algorithmic secrets.

Programming as Contracting

A class is a contract between an abstraction and its clients. Strongly typed languages can detect violations of this contract at compile time.

4.5 Visibility Levels

Visibility	Accessible By	Typical Use
Public	All clients	Stable operations intended for general use
Protected	Class itself and subclasses	Extension points for inheritance
Private	Class itself only	Representation and helper methods
Package	Classes in the same package/module	Internal collaboration within a subsystem

4.5.1 Friendship and Package Access

Some languages provide mechanisms that intentionally weaken encapsulation:

- C++ friendship allows selected classes or functions to access private or protected members.
- Java has package access, where classes in the same package can access package-visible members.

Tradeoff

Friendship and package access can simplify tightly integrated implementations, but they increase coupling and should be used carefully. Every extra visibility privilege expands the set of code that can depend on representation details.

4.6 Class Lifecycle Semantics

For simple classes, understanding each operation independently may be enough. For more interesting classes, the meaning of an operation depends on the object's lifecycle.

Examples:

- `move()` may be valid only after a display item has been created and attached to a view;
- `ship()` may be valid only after an order has been paid;
- `close()` may be valid only if a connection is open.

Such behavior is often represented using state machine diagrams.

5 Relationships Among Classes

5.1 Overview

Classes rarely exist in isolation. Their relationships form the class structure of the design.

There are three major class relationships:

1. **association**: semantic dependency;
2. **inheritance**: generalization/specialization, or “is-a”;
3. **aggregation**: whole/part, or “part-of”.

Relationship	Meaning	Example
Association	Classes are semantically connected	Ladybug protects flower
Inheritance	One class is a specialized kind of another	Rose is a flower
Aggregation	One class represents a whole containing parts	Flower has petals
Dependency	One element depends on another and may be affected by changes	Service depends on repository interface

5.2 Association

An **association** is the most general and semantically weakest class relationship. It says that classes are meaningfully connected but does not necessarily specify ownership, inheritance, or implementation.

5.2.1 When to Use Association

Use association during analysis when you know two classes are related but have not yet decided whether the relationship should become aggregation, dependency, inheritance, or a more precise implementation mechanism.

Example:

- **Vehicle** is associated with **Wheel**.
- Initially this may be drawn as an association.
- Later it may be refined into aggregation because wheels are parts of a vehicle.

5.2.2 Multiplicity

Multiplicity describes how many instances of one class may relate to instances of another.

Multiplicity	Meaning	Example
One-to-one	Each object on one side relates to exactly one on the other	Sale and credit card transaction
One-to-many	One object relates to many objects	Vehicle and wheels
Many-to-many	Many objects relate to many objects	Customers and salespeople

5.2.3 Association Design Mistakes

Common Association Mistakes

- Treating every association as bidirectional when only one-way navigation is needed.
- Forgetting multiplicity and leaving cardinality ambiguous.
- Using association when the domain relationship is actually whole/part.
- Turning every association into a direct field reference, causing unnecessary coupling.

5.3 Inheritance

Inheritance expresses a generalization/specialization relationship among classes.

Definition

Inheritance is a relationship among classes wherein one class shares the structure and/or behavior defined in one or more other classes.

The class inherited from is the **superclass**. The inheriting class is the **subclass**.

5.3.1 The Is-A Litmus Test

Inheritance Litmus Test

Given classes A and B, A should inherit from B only if every A is genuinely a kind of B.

Good examples:

- **ElectricalData** is a kind of **TelemetryData**;
- **Circle** is a kind of **DisplayItem**;
- **SavingsAccount** is a kind of **BankAccount**.

Bad examples:

- **Car** should not inherit from **Engine**; a car has an engine.

- `Stack` should not necessarily inherit from `ArrayList`; it may use a list internally.
- `UserService` should not inherit from `DatabaseConnection`; it depends on one.

5.3.2 Inheritance for Extension and Restriction

Use	Meaning	Example
Extension	Subclass adds behavior or state	<code>GuardedQueue</code> adds thread-safe operations
Restriction	Subclass constrains superclass behavior	<code>UnselectableDisplayItem</code> prohibits selection
Combination	Subclass both adds and constrains behavior	A restricted account with added audit logging

5.4 Abstract and Concrete Classes

- A **concrete class** is intended to have direct instances.
- An **abstract class** captures common behavior but is incomplete or too general to instantiate directly.
- A **leaf class** is a concrete class at the bottom of an inheritance hierarchy.

Listing 3: Abstract class and polymorphic subclasses

```

abstract class DisplayItem {
    private Point center;

    public Point location() {
        return center;
    }

    public abstract void draw();
    public abstract void moveTo(Point newCenter);
}

final class Circle extends DisplayItem {
    private double radius;

    @Override
    public void draw() {
        // draw a circle using center and radius
    }

    @Override
    public void moveTo(Point newCenter) {
        // update position and redraw as needed
    }
}

```

5.5 Polymorphism

Polymorphism allows the same operation name to be used for objects of different subclasses, with each object responding according to its own class.

Definition

Polymorphism means that a name may denote instances of many different classes related by a common superclass, and each object may respond to the same operation in a class-specific way.

Listing 4: Polymorphic dispatch removes type-based conditionals

```
List<DisplayItem> items = List.of(
    new Circle(),
    new Rectangle(),
    new Triangle()
);

for (DisplayItem item : items) {
    item.draw(); // dynamic dispatch selects the correct implementation
}
```

5.5.1 Why Polymorphism Matters

Without polymorphism, developers often write large `switch` or `if` statements based on object type.

Listing 5: Type-switch smell

```
void draw(DisplayItem item) {
    if (item instanceof Circle) {
        drawCircle((Circle) item);
    } else if (item instanceof Rectangle) {
        drawRectangle((Rectangle) item);
    } else if (item instanceof Triangle) {
        drawTriangle((Triangle) item);
    }
}
```

This is brittle because adding a new shape requires modifying every type-switch location. Polymorphism moves the behavior to the class that owns the knowledge.

Interview Insight

A type switch over subclasses is often a sign that polymorphism has not been used effectively. The object should usually know how to perform its own behavior.

5.6 Overloading Versus Polymorphism

Concept	Meaning	Example
---------	---------	---------

Overloading	Same name, different parameter signatures, usually resolved at compile time	<code>print(int)</code> and <code>print(String)</code>
Overriding	Subclass provides a new implementation of superclass operation	<code>Circle.draw()</code> overrides <code>DisplayItem.draw()</code>
Polymorphism	Runtime object determines which overridden method executes	<code>item.draw()</code> calls subclass method
Late binding	Method implementation selected at runtime	Java virtual dispatch
Early binding	Method implementation selected at compile time	Static method call or non-virtual call

5.7 Dynamic Dispatch

In a polymorphic call, the runtime system selects the correct method implementation based on the actual object, not merely the static type of the variable.

Conceptually:

1. The client sends a message such as `draw()`.
2. The runtime examines the object's actual class.
3. The closest applicable implementation in the inheritance hierarchy is selected.
4. That method executes.

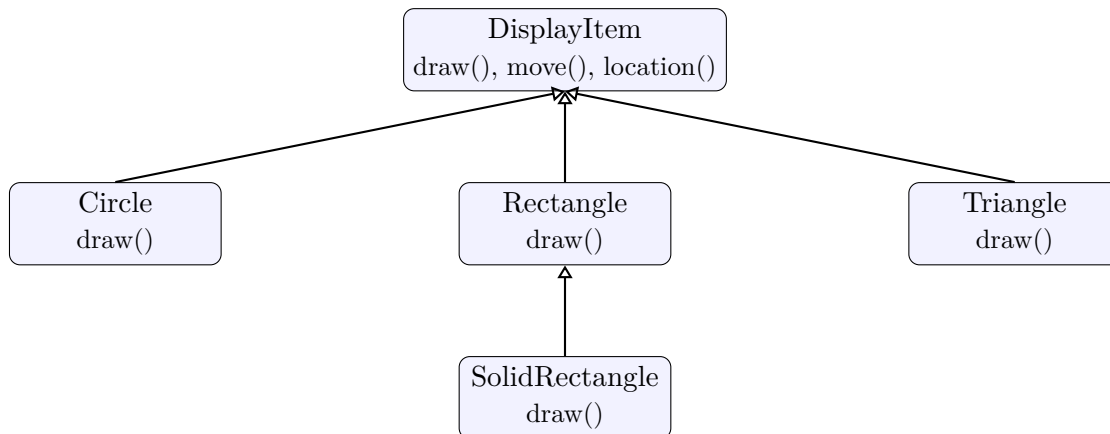


Figure 3: A superclass protocol enables polymorphic behavior across subclasses.

5.8 Single Inheritance

In **single inheritance**, each subclass has exactly one direct superclass.

Benefits:

- simpler mental model;
- fewer name collision problems;
- easier method resolution;
- often adequate for strict taxonomies.

Tradeoffs:

- may force an artificial hierarchy;
- cannot directly represent overlapping orthogonal classifications;
- can encourage duplicated behavior if common behavior does not fit one clean superclass.

5.9 Multiple Inheritance

In **multiple inheritance**, a subclass may inherit from more than one superclass.

It is useful when a class belongs to overlapping categories. For example, an asset may be both `InsurableItem` and `InterestBearingItem`.

5.9.1 Problems in Multiple Inheritance

Problem	Meaning	Possible Handling
Name collision	Two superclasses define members with the same name	Reject, merge, or require qualification
Repeated inheritance	Same ancestor appears through multiple paths	Reject, duplicate, or share ancestor subobject
Ambiguous behavior	Subclass inherits conflicting semantics	Override explicitly or redesign hierarchy
High coupling	Subclass depends on multiple parent internals	Prefer composition or interfaces when possible

5.9.2 Mixins

A **mixin** is a class-like abstraction whose purpose is to add a focused behavior to another class through inheritance or a similar language mechanism.

Examples:

- `Comparable`;
- `Serializable`;
- `Auditable`;
- `InsurableItem`;
- `InterestBearingItem`.

Modern Engineering Interpretation

Many modern languages avoid full multiple inheritance of implementation but support interfaces, traits, default methods, or composition-based mixins to gain reuse while reducing ambiguity.

5.10 Inheritance Versus Encapsulation

Inheritance creates coupling between superclass and subclass. A subclass often depends on superclass structure, behavior, and protected extension points.

Inheritance Hazard

Inheritance can expose implementation secrets. To understand a subclass, developers may need to understand its entire superclass chain. Deep hierarchies can therefore become fragile and difficult to reason about.

Use inheritance when:

- the relationship is truly **is-a**;
- the superclass defines a stable abstraction;
- subclasses preserve the superclass contract;
- polymorphism is valuable.

Avoid inheritance when:

- the relationship is merely **has-a** or **uses-a**;
- the subclass would violate superclass invariants;
- reuse is the only motivation;
- a change in the superclass would frequently break subclasses.

5.11 Aggregation Among Classes

Aggregation among classes represents whole/part relationships that will appear among their instances.

Example:

- `TemperatureController` is the whole.
- `Heater` is one part.
- `TemperatureRamp` may be another collaborating part.

5.11.1 Containment by Value and by Reference

Containment Form	Meaning	Lifecycle Implication
Containment by value	Part is created and destroyed with the whole	Strong lifetime coupling
Containment by reference	Whole refers to part indirectly	Part may exist independently
Conceptual aggregation	Whole/part meaning without physical containment	Ownership or membership may be indirect

Aggregation Litmus Test

Use aggregation if and only if there is a genuine whole/part relationship between the objects represented by the classes.

5.12 Dependencies

A **dependency** indicates that one element depends on another. If the supplier changes, the dependent element may be affected.

Dependencies often appear:

- in architecture diagrams between packages or components;
- in implementation diagrams between modules;
- when one class calls another without owning it;
- when one service depends on an interface.

Listing 6: Dependency through an interface reduces coupling

```
interface PaymentGateway {
    PaymentResult charge(Money amount, Card card);
}

final class CheckoutService {
    private final PaymentGateway gateway;

    CheckoutService(PaymentGateway gateway) {
        this.gateway = gateway;
    }

    PaymentResult checkout(Order order, Card card) {
        return gateway.charge(order.total(), card);
    }
}
```

6 The Interplay of Classes and Objects

6.1 Static Classes, Dynamic Objects

Classes and objects are separate but tightly related:

- every object is an instance of some class;
- every class has zero or more instances;
- classes are usually static in a program;
- objects are usually created and destroyed dynamically.

In an air traffic control system:

- classes such as `Plane`, `Runway`, `FlightPlan`, and `AirSpace` are relatively stable;
- objects representing specific planes and flight plans appear and disappear frequently;
- system correctness depends on stable class semantics, such as “two planes should not occupy the same airspace.”

6.2 Key Abstractions and Mechanisms

During analysis and early design, developers have two primary tasks:

1. identify the classes that form the vocabulary of the problem domain;
2. invent structures whereby sets of objects collaborate to satisfy system requirements.

Definitions

Key abstractions are the important classes and objects that form the vocabulary of the problem domain. **Mechanisms** are structures of collaborating objects that provide behavior satisfying system requirements.

6.3 Outside View Versus Inside View

View	Focus	Typical Design Questions
Outside view	Logical framework, interfaces, responsibilities, collaborations	What services are promised? Who talks to whom? What roles exist?
Inside view	Representation, algorithms, data structures, packaging	How is state stored? Which algorithm is used? Where does code live?

Process Insight

Start with the outside view. Stabilize responsibilities, protocols, and collaborations before committing to representation details.

7 Building Quality Classes and Objects

7.1 Iterative Nature of OO Design

High-quality classes rarely emerge perfectly on the first attempt. Class design is incremental and iterative. Designers refine abstractions as they build clients, discover missing responsibilities, and observe repeated patterns.

However, refinement has costs:

- recompilation or redeployment;
- migration of clients;
- increased cognitive load;
- risk of breaking architectural integrity.

7.2 Quality Criteria for Abstractions

The quality of a class or object can be judged by five criteria:

1. coupling;
2. cohesion;
3. sufficiency;
4. completeness;
5. primitiveness.

7.3 Coupling

Coupling measures the strength of dependency between abstractions.

Goal

Design with the weakest coupling that still allows objects to collaborate effectively.

Strong coupling makes a class harder to:

- understand independently;
- test in isolation;
- reuse in a different context;
- change without affecting clients.

7.3.1 Coupling and Inheritance

Inheritance introduces significant coupling because subclasses depend on superclass contracts and sometimes superclass internals. This creates a tension:

- weak coupling is desirable;
- inheritance helps exploit commonality;
- overusing inheritance produces fragile hierarchies.

7.4 Cohesion

Cohesion measures how strongly the elements of one class or module belong together.

Cohesion Type	Meaning	Quality
Coincidental cohesion	Unrelated responsibilities are grouped together	Very poor
Functional cohesion	All elements contribute to one well-bounded behavior	Excellent

Cohesion Rule

A class should model the abstraction, the whole abstraction, and nothing but the abstraction.

7.5 Sufficiency

A class is **sufficient** when it captures enough characteristics to permit meaningful and efficient interaction.

Example: a **Set** abstraction with **remove()** but no **add()** is insufficient. Clients cannot use it meaningfully.

7.6 Completeness

A class is **complete** when its interface captures all meaningful characteristics of the abstraction.

Completeness is broader than sufficiency:

- sufficiency asks whether the class has enough to be useful;

- completeness asks whether the class covers the abstraction well for general clients.

Tradeoff

Completeness can be overdone. Too many operations overwhelm clients and create a fat interface. Prefer complete abstractions built from primitive operations rather than bloated convenience APIs.

7.7 Primitiveness

An operation is **primitive** if it can be efficiently implemented only with access to the underlying representation of the abstraction.

Example:

- `Set.add(item)` is primitive because it requires representation access.
- `Set.addAll(a,b,c,d)` may not be primitive because it can call `add()` repeatedly.

Primitive Operation Rule

Expose operations that need privileged access to representation. Build nonprimitive convenience behavior on top of primitive operations when possible.

7.8 Choosing Operations

Crafting a class interface is difficult because operations define the class contract. Good operations are:

- small and well-defined;
- meaningful to clients;
- aligned with the object's responsibilities;
- stable across likely representation changes;
- neither too coarse nor too fragmented.

7.8.1 Fine-Grained Versus Coarse-Grained Operations

Choice	Benefit	Risk
Fine-grained methods	Easier to override, test, compose, and reason about	Too many methods may fragment the interface
Coarse-grained methods	Simpler public surface for clients	Methods may become large, rigid, and hard to reuse

7.8.2 Where Should a Behavior Live?

When deciding which class should own a behavior, consider:

- **Reusability:** would this behavior be useful in more than one context?
- **Complexity:** how hard is the behavior to implement?
- **Applicability:** is the behavior relevant to this abstraction?

- **Implementation knowledge:** does the behavior require internal details of a class?

Responsibility Assignment Rule

Place behavior with the class that has the information needed to perform it, unless doing so creates unacceptable coupling or violates cohesion.

7.9 Time and Space Semantics

After defining what an operation does, specify its performance expectations:

- best-case time;
- average-case time;
- worst-case time;
- storage required;
- synchronization behavior;
- side effects.

Example: a flight-plan repository may optimize for fast search, fast insertion, or fast deletion, but usually cannot optimize all equally.

7.10 Choosing Relationships

Relationship choices determine visibility, coupling, and collaboration style.

7.10.1 Law of Demeter

The **Law of Demeter** says that a method should communicate only with a limited set of closely related objects and should not depend on the internal structure of distant objects.

Informally: **only talk to your immediate friends.**

Listing 7: Law of Demeter violation

```
class OrderService {
    Money shippingCost(Order order) {
        // Reaches through several objects and depends on internal structure.
        return order.getCustomer()
            .getAddress()
            .getRegion()
            .getShippingRate();
    }
}
```

Listing 8: Improved responsibility placement

```
class Order {
    Money shippingCost(ShippingPolicy policy) {
        return policy.costFor(destinationRegion());
    }

    private Region destinationRegion() {
        return customer.shippingRegion();
    }
}
```

```
}
}
}
```

Law of Demeter Tradeoff

The Law of Demeter reduces coupling and representation leaks, but applying it mechanically can create too many forwarding methods. Use it as a design smell detector, not as a rigid rule.

7.10.2 Wide-Shallow Versus Narrow-Deep Inheritance

Hierarchy Shape	Benefit	Risk
Wide and shallow	Looser coupling; classes easier to understand independently	May duplicate common behavior
Narrow and deep	Exploits commonality; smaller subclasses	Harder to understand; changes ripple through hierarchy
Balanced	Uses inheritance where commonality is real	Requires careful classification

7.11 Inheritance, Aggregation, and Dependency Decision Guide

Relationship	Use When	Example
Inheritance	Every instance of subclass is also an instance of superclass	<code>Circle</code> is a <code>Shape</code>
Aggregation	One object is part of another	<code>Car</code> has <code>Wheels</code>
Dependency	One class uses another but does not own it	<code>CheckoutService</code> uses <code>PaymentGateway</code>
Association	Domain concepts are semantically connected	<code>Doctor</code> treats <code>Patient</code>

Classic Mistake

Do not use inheritance merely for code reuse. If the domain relationship is not “is-a,” prefer composition, aggregation, delegation, or dependency through an interface.

7.12 Mechanisms and Visibility

A mechanism is a collaboration structure that satisfies a requirement. Designing mechanisms means deciding where knowledge belongs.

Example question: a manufacturing lot enters a room, and the room manager must be notified.

Possible designs:

- operation on `Room`: `room.enter(lot);`
- operation on `Lot`: `lot.enter(room);`
- operation on both, coordinated by a controller;

- domain event: `LotEnteredRoom` handled by a manager service.

Visibility Question

Ask: who must know whom? Each visibility decision becomes a dependency. Good mechanisms place knowledge where it is cohesive and minimize unnecessary coupling.

7.13 Choosing Implementations

Only after stabilizing the outside view should the designer focus on implementation.

Implementation decisions include:

- representation of state;
- algorithms for operations;
- caching versus computation;
- packaging into modules;
- performance and resource tradeoffs.

7.13.1 Representation as a Secret

The representation of a class should almost always be hidden.

Example: a `Cone` class may compute volume on demand or store cached volume.

Choice	Benefit	Cost
Compute volume on demand	Saves memory; avoids stale cache	Slower repeated calls
Store volume as field	Faster reads	Extra memory; must maintain consistency

Listing 9: Representation can change without changing the public contract

```
final class Cone {
    private final double radius;
    private final double height;

    Cone(double radius, double height) {
        this.radius = radius;
        this.height = height;
    }

    public double volume() {
        return Math.PI * radius * radius * height / 3.0;
    }
}
```

7.14 Packaging

Packaging determines where classes and objects are declared and how modules depend on each other.

Good packages are:

- functionally cohesive;
- loosely coupled;
- stable at the public interface;
- explicit about what is exported and what is internal;
- organized around likely changes.

Packaging is influenced by:

- reuse;
- security;
- testing;
- documentation;
- deployment;
- team ownership.

8 UML and Diagram Interpretation

8.1 Class Diagrams

A class diagram communicates static structure:

- class names;
- attributes;
- operations;
- visibility;
- inheritance;
- association;
- aggregation;
- dependencies;
- multiplicities.

Use class diagrams when you need to explain the vocabulary and relationships of a domain or design.

8.2 Object Diagrams

An object diagram shows specific runtime instances and links among them. It is useful for explaining:

- object identity;
- aliasing;
- snapshots of state;
- concrete examples of class relationships.

8.3 State Machine Diagrams

A state machine diagram communicates lifecycle behavior:

- states;
- events;
- valid transitions;
- actions;
- invalid operation sequences.

Use it for objects whose behavior depends heavily on event ordering.

8.4 Common UML Mistakes

Diagram Mistakes

- Drawing inheritance for **has-a** relationships.
- Omitting multiplicity on associations.
- Exposing all fields as public attributes.
- Modeling implementation details during early analysis.
- Using bidirectional associations by default.
- Creating diagrams that show everything but communicate nothing.

9 Key Takeaways

1. An object has state, behavior, and identity.
2. A class defines the common structure, behavior, and semantics of similar objects.
3. State is not just data; it is the cumulative result of behavior over time.
4. Behavior is expressed through operations, methods, or messages.
5. Identity is distinct from names, values, and references.
6. Objects collaborate through links and aggregation.
7. Classes relate through association, inheritance, aggregation, and dependencies.
8. Inheritance should model true generalization/specialization, not mere code reuse.
9. Polymorphism removes brittle type-based conditionals.
10. Aggregation models whole/part relationships.
11. Good abstractions are loosely coupled, cohesive, sufficient, complete, and primitive.
12. Design begins with outside views: responsibilities, contracts, and collaborations.
13. Implementation details should remain hidden behind stable interfaces.

10 Common Pitfalls

10.1 Pitfall 1: Confusing Object Identity with Equality

Two objects can have the same state but different identities. Comparing references when logical equality is intended causes bugs.

10.2 Pitfall 2: Exposing Representation

Public fields make every client dependent on internal structure. Use private or protected representation and public operations.

10.3 Pitfall 3: Using Inheritance for Code Reuse Alone

Inheritance should express “is-a.” If the goal is only reuse, composition or delegation is usually safer.

10.4 Pitfall 4: Creating Fat Interfaces

A class that exposes too many unrelated operations becomes hard to learn, test, evolve, and reuse.

10.5 Pitfall 5: Ignoring Lifecycle Semantics

Some operations are valid only in certain states. Ignoring lifecycle rules leads to invalid transitions and inconsistent objects.

10.6 Pitfall 6: Overusing Bidirectional Links

Bidirectional associations increase coupling and make object graphs harder to maintain.

10.7 Pitfall 7: Forgetting Synchronization

Objects designed for sequential use may fail under concurrency. Decide whether objects are sequential, guarded, or concurrent.

10.8 Pitfall 8: Violating the Law of Demeter

Long chains of navigation expose internal structure and make clients fragile.

11 Interview Questions

11.1 Objects and Classes

Question: What are the three essential characteristics of an object?

Concise Answer: An object has state, behavior, and identity. State describes what it currently knows, behavior describes what it can do or how it reacts, and identity distinguishes it from all other objects even if their states are equal.

Question: What is the difference between a class and an object?

Concise Answer: A class is an abstraction that defines common structure, behavior, and semantics. An object is a concrete runtime instance of a class with its own state and identity.

Question: Why is object identity not the same as a variable name?

Concise Answer: A variable is only a reference or handle through which an object may be accessed. Multiple variables may refer to the same object, and an object may outlive or become disconnected from a particular variable name.

Question: Why should object state usually be encapsulated?

Concise Answer: Encapsulation prevents clients from depending on internal representation. This allows the class to change data structures or invariants without breaking external code that uses the public contract.

11.2 Behavior and Operations

Question: What is the difference between a modifier and a selector?

Concise Answer: A modifier changes object state, while a selector reads state without changing it. For example, `deposit()` is a modifier and `balance()` is a selector.

Question: Why does behavior depend on state?

Concise Answer: The same operation may produce different results depending on previous operations. For example, a vending machine responds differently to a selection depending on whether sufficient money has been inserted.

Question: What is a protocol?

Concise Answer: A protocol is the set of operations an object exposes to clients. It defines the allowable behavior clients may request from that object.

11.3 Relationships Among Objects

Question: What is a link between objects?

Concise Answer: A link is a physical or conceptual connection that allows one object to navigate to another or request one of its services. It supports object collaboration.

Question: What is aggregation between objects?

Concise Answer: Aggregation is a whole/part relationship. It indicates that one object is an aggregate and another object is one of its parts, such as an airplane and its engines.

Question: What is the difference between a controller, server, and proxy object?

Concise Answer: A controller initiates operations on other objects, a server is operated on by others, and a proxy both receives operations and represents or delegates to another real-world or remote object.

Question: Why does visibility matter in object relationships?

Concise Answer: An object can send a message only to objects it can see. Visibility choices determine coupling, access scope, and how object collaborations are implemented.

11.4 Class Relationships

Question: What are the three main class relationships?

Concise Answer: The main class relationships are association, inheritance, and aggregation. Association models semantic dependency, inheritance models “is-a,” and aggregation models whole/part.

Question: What is the inheritance litmus test?

Concise Answer: Use inheritance only when every instance of the subclass is truly an instance of the superclass. If the relationship is not “is-a,” use aggregation, composition, delegation, or dependency instead.

Question: Why is inheritance both useful and risky?

Concise Answer: Inheritance supports reuse, classification, and polymorphism, but it tightly couples subclasses to superclasses. Changes in the superclass can ripple through the hierarchy and expose implementation details.

Question: What problem does polymorphism solve?

Concise Answer: Polymorphism eliminates large type-based conditional logic by letting each object respond to the same operation according to its own class. This improves extensibility because new subclasses can be added with less modification to existing client code.

Question: What is the difference between overloading and overriding?

Concise Answer: Overloading defines multiple operations with the same name but different parameter signatures. Overriding replaces or specializes a superclass operation in a subclass and is central to runtime polymorphism.

Question: What is multiple inheritance and why is it difficult?

Concise Answer: Multiple inheritance allows a class to inherit from more than one superclass. It is difficult because it can create name collisions, repeated inheritance, ambiguous behavior, and stronger coupling.

11.5 Quality of Abstractions

Question: What is coupling?

Concise Answer: Coupling is the strength of dependency between abstractions. Lower coupling is preferred because it improves understandability, testability, reuse, and changeability.

Question: What is cohesion?

Concise Answer: Cohesion is the degree to which the elements of a class or module belong together. High cohesion means the class has one well-bounded purpose.

Question: What does it mean for a class to be sufficient?

Concise Answer: A sufficient class provides enough operations and properties for clients to use it meaningfully and efficiently. A set without an add operation would be insufficient.

Question: What does it mean for a class to be complete?

Concise Answer: A complete class covers the meaningful characteristics of its abstraction for general use. Completeness must be balanced against interface bloat.

Question: What does primitive mean in class design?

Concise Answer: A primitive operation is one that needs direct access to the object's representation to be implemented efficiently. Nonprimitive convenience operations can often be built from primitive ones.

Question: What is the Law of Demeter?

Concise Answer: The Law of Demeter says a method should communicate only with a limited set of immediate collaborators and should not depend on distant object structure. It reduces coupling and representation leakage.

12 Engineering Insights

12.1 Objects Are Responsibility Holders, Not Data Bags

A class with fields and trivial getters/setters but no meaningful behavior is often just a disguised record. Good OO design assigns responsibilities to objects and lets behavior live with the data and invariants it needs.

12.2 Polymorphism Supports Open/Closed Design

When clients depend on a superclass or interface, new subclasses can often be added without changing client logic. This supports the Open/Closed Principle: open for extension, closed for modification.

12.3 Encapsulation Enables Representation Change

A stable interface allows internal data structures to evolve. This is essential for performance tuning, refactoring, and long-term maintenance.

12.4 Inheritance Is a Semantic Commitment

Inheritance is not just a convenience mechanism. It says something strong about substitutability, classification, and expected behavior. A bad superclass/subclass relationship can damage the entire design.

12.5 Object Design Is Also Concurrency Design

Once multiple active objects exist, message passing involves synchronization decisions. Objects must explicitly be designed as sequential, guarded, or concurrent.

12.6 Mechanisms Are the Heart of OO Architecture

Classes define vocabulary, but mechanisms define behavior. A system succeeds when objects collaborate through clear, cohesive mechanisms that satisfy requirements without excessive coupling.

13 Connections to Other Topics

13.1 Connection to Abstraction

Classes and objects are the main units of abstraction in object-oriented design. They hide irrelevant details while exposing meaningful behavior.

13.2 Connection to Encapsulation

Encapsulation protects object state and implementation secrets. It is implemented through visibility control, stable interfaces, and representation hiding.

13.3 Connection to Modularity

Classes group related state and behavior. Packages and modules group related classes. Good modularity requires high cohesion and low coupling at both levels.

13.4 Connection to SOLID Principles

SOLID Principle	Connection
Single Responsibility	A cohesive class should have one well-bounded reason to change.

Open/Closed	Polymorphism allows extension through new subclasses or implementations.
Liskov Substitution	Inheritance is valid only when subclasses preserve superclass contracts.
Interface Segregation	Avoid fat interfaces; clients should depend only on operations they need.
Dependency Inversion	Depend on abstractions rather than concrete implementation classes.

13.5 Connection to Design Patterns

Many design patterns refine the relationships in this chapter:

- Strategy uses polymorphism to vary behavior without conditionals.
- Composite uses aggregation and polymorphism to treat parts and wholes uniformly.
- Proxy uses an object to represent another object.
- Adapter changes an interface while preserving client expectations.
- Observer manages links and dependencies between subjects and observers.
- Factory centralizes object creation and hides concrete class selection.

13.6 Connection to System Design

At system scale:

- classes become services, components, modules, or bounded contexts;
- object links become API calls, events, or message queues;
- visibility becomes dependency management;
- synchronization becomes distributed coordination;
- encapsulation becomes service boundary design.

14 Practical Applications

14.1 Domain Modeling

Use classes to capture domain vocabulary and object relationships:

- `Customer`, `Order`, `Invoice`, `Payment`;
- associations such as customer places order;
- aggregation such as order contains order lines;
- inheritance only when domain types form a true hierarchy.

14.2 API Design

A public class interface is an API contract. Good APIs:

- expose behavior, not representation;
- avoid unnecessary setters;
- preserve invariants;

- make invalid states hard to represent;
- keep operations cohesive and primitive.

14.3 Refactoring

Chapter 3 concepts guide refactoring:

- replace type code with subclasses or strategy;
- extract class when cohesion is low;
- move method when behavior belongs elsewhere;
- hide delegate to reduce Law of Demeter violations;
- replace inheritance with delegation when “is-a” is false.

14.4 Testing

Good object design improves testing:

- low coupling enables mocking and stubbing;
- high cohesion makes behavior easier to test;
- clear state machines support lifecycle tests;
- encapsulation prevents tests from depending on internals.

14.5 Architecture Documentation

Use:

- class diagrams for static structure;
- object diagrams for runtime snapshots;
- sequence diagrams for object collaboration;
- state machines for lifecycle behavior;
- package/component diagrams for dependencies.

15 Summary Cheat Sheet

15.1 Core Definitions

Term	Meaning
Object	Entity with state, behavior, and identity
Class	Set of objects sharing common structure, behavior, and semantics
State	Properties plus current values
Behavior	Actions and reactions through state changes and message passing
Identity	Property that distinguishes one object from all others
Protocol	Set of operations available to clients
Role	Client-relative mask or contract an object supports
Responsibility	Service an object provides as part of a role
Link	Connection allowing one object to request services from another

Aggregation	Whole/part relationship
Association	Semantic dependency between classes
Inheritance	Generalization/specialization relationship
Polymorphism	Same operation name, class-specific behavior at runtime
Coupling	Strength of dependency between abstractions
Cohesion	Degree to which elements of an abstraction belong together
Sufficiency	Interface has enough to be useful
Completeness	Interface covers the meaningful abstraction
Primitiveness	Operations expose only essential representation-dependent behavior

15.2 Relationship Decision Table

Question	Relationship	Example
Is A a kind of B?	Inheritance	Circle is a Shape
Does A contain or own B as a part?	Aggregation	Order has OrderLines
Does A simply use B?	Dependency	Service uses Repository
Are A and B meaningfully connected?	Association	Doctor treats Patient
Does A request services from B at runtime?	Link	Controller calls Valve

15.3 Design Heuristics

1. Start with domain vocabulary, not implementation details.
2. Hide state; expose behavior.
3. Prefer cohesive classes with clear responsibilities.
4. Use inheritance only for true “is-a” relationships.
5. Prefer composition or delegation for reuse.
6. Use polymorphism to replace type-based conditionals.
7. Keep interfaces sufficient, complete, and primitive.
8. Minimize visibility to reduce coupling.
9. Design lifecycles explicitly for stateful objects.
10. Treat concurrency as part of object semantics.

15.4 Exam Memory Hooks

- **Object = state + behavior + identity.**
- **Class = common structure + common behavior + common semantics.**
- **Inheritance = is-a.**
- **Aggregation = part-of.**
- **Association = semantic connection.**

- Polymorphism = same request, different class-specific response.
- Encapsulation = protect representation behind contract.
- Good abstraction = low coupling + high cohesion + sufficient + complete + primitive.